EXPERIMENT 4

AIM

To implement the **K-Nearest Neighbors (KNN)** algorithm for classification and evaluate the model using different performance metrics.

THEORY

1. Dataset Source

Dataset Name: Iris Dataset

Source: Kaggle

Link:
https://www.kaggle.com/datasets/uciml/iris

2. Dataset Overview

The **Iris dataset** contains measurements of iris flowers belonging to three distinct species. Using these measurements, the objective is to classify each flower into the correct species.

Dataset Characteristics

- **Total Samples:** 150
- **Number of Input Features:** 4
- **Number of Classes:** 3
- **Samples per Class:** 50
- **Problem Type:** Multiclass Classification
- **Dataset Nature:** Balanced dataset

Input Features (Independent Variables)

The dataset includes four numerical attributes:

1. **Sepal Length (cm)**
2. **Sepal Width (cm)**
3. **Petal Length (cm)**
4. **Petal Width (cm)**

All these variables represent continuous measurements of flower parts.

Target Variable

The target variable represents the **species of the iris flower**, which can take three possible values:

- **Iris-setosa**
- **Iris-versicolor**
- **Iris-virginica**

Important Observations

- **Iris-setosa** is easily separable from the other two species.
- **Versicolor** and **Virginica** have partially overlapping feature distributions.
- The dataset is well-structured and **does not contain missing values**.
- Due to its simplicity and balance, it is commonly used to illustrate classification techniques.

3. Mathematical Formulation of KNN

The **K-Nearest Neighbors (KNN)** algorithm is a **non-parametric and instance-based learning method** used for classification and regression tasks.

Unlike other algorithms, KNN does not build an explicit model during training. Instead, it stores the training data and performs prediction based on similarity between data points.

Step 1: Selecting the Value of K

A parameter **K** is chosen to represent the number of neighboring samples used for classification.

Step 2: Distance Measurement

The similarity between data points is measured using a distance metric. The most common metric is **Euclidean Distance**

Where:

- xixi = feature value of the test sample
- yiyi = feature value of the training sample
- nn = number of features

Step 3: Finding Nearest Neighbors

Distances between the test sample and all training samples are computed and sorted in ascending order. The **K closest neighbors** are then selected.

Step 4: Classification Decision

The predicted class is determined using **majority voting** among the selected neighbors

Where:

- $y1,y2,...,yk$y1,y2,...,yk represent the classes of the K nearest neighbors.

4. Limitations of KNN

Although KNN is simple and effective, it has several drawbacks:

1. Prediction time increases for **large datasets** because the algorithm compares each test sample with all training samples.
2. The algorithm is highly dependent on **feature scaling** because distance calculations are used.
3. It is sensitive to **noise and outliers** in the data.
4. Performance can degrade in **high-dimensional datasets** due to the curse of dimensionality.
5. Choosing an inappropriate **K value** may lead to poor classification.
6. It may produce biased results when working with **imbalanced datasets**.

5. Methodology / Workflow

The experiment was carried out using the following procedure.

Step 1: Data Acquisition

The Iris dataset was obtained from Kaggle and imported into the Google Colab environment for analysis.

Step 2: Data Preprocessing

Several preprocessing steps were applied before training the model:

- The **Id column** was removed since it does not contribute to prediction.
- Input features (**X**) and target variable (**y**) were separated.
- Feature normalization was applied using **StandardScaler**:

Where:

- xx = original feature value
- μμ = mean of the feature
- σσ = standard deviation

This step ensures that all features contribute equally to distance calculations.

Step 3: Train-Test Splitting

The dataset was divided into two subsets:

- **Training Data:** 80%
- **Testing Data:** 20%

Stratified sampling was used to maintain the same class distribution in both sets.

Step 4: Model Training

A **KNN classifier** was implemented using the **scikit-learn library**.
The initial value of the hyperparameter was selected as:

K=5K=5

Step 5: Model Evaluation

The model performance was evaluated using the following metrics:

- **Accuracy**
- **Confusion Matrix**
- **Classification Report** (Precision, Recall, F1-score)
- **Accuracy vs K Graph**

6. Performance Analysis

Confusion Matrix Interpretation

The results indicate that:

- **Iris-setosa** was classified with **100% accuracy**.
- **Iris-versicolor** was also correctly classified in most cases.
- **Iris-virginica** achieved approximately **80% recall**, with a few samples misclassified as versicolor.

Overall, the classifier performed very well on the dataset.

The slight confusion between **versicolor and virginica** occurs because their feature values overlap in the dataset.

Precision, Recall and F1 Score

- **Macro Average F1 Score ≈ 0.93**
- **Weighted Average F1 Score ≈ 0.93**

These values indicate that the classifier performs consistently across all three classes.

7. Hyperparameter Tuning

The primary hyperparameter in KNN is the **number of neighbors (K)**.

Different K values ranging from **1 to 20** were evaluated by plotting an **Accuracy vs K graph**.

Observations

- **Very small K values (e.g., K = 1)** may lead to **overfitting** because the model becomes sensitive to noise.
- **Large K values (e.g., above 15)** may oversmooth decision boundaries and lead to **underfitting**.
- The model maintained stable accuracy between **K = 3 and K = 9**.

Final Selected Value

K=5K=5

This value was selected because it:

- Provides **high and stable accuracy (~93–96%)**
- Reduces the effect of noise
- Avoids tie situations due to an **odd value of K**
- Maintains a good balance between **bias and variance**

Code:

```
# ======================================
# Step 1: Import Required Libraries
# ======================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

sns.set_style("whitegrid")

# ======================================
# Step 2: Load Dataset
# ======================================
from google.colab import files
uploaded = files.upload()

df = pd.read_csv("Iris.csv")

print("First 5 Rows:\n")
print(df.head())

print("\nDataset Shape:", df.shape)
```

```
# ======================================
# Step 3: Data Preprocessing
# ======================================

# Drop Id column if present
if "Id" in df.columns:
    df.drop("Id", axis=1, inplace=True)

# Separate Features and Target
X = df.drop("Species", axis=1)
y = df["Species"]

print("\nFeature Columns:", X.columns.tolist())
print("Target Classes:", y.unique())

# ======================================
# Step 4: Feature Scaling (Important for KNN)
# ======================================
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ======================================
# Step 5: Train-Test Split
# ======================================
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])

# ======================================
# Step 6: Train KNN Model
# ======================================
k = 5
knn = KNeighborsClassifier(n_neighbors=k)

knn.fit(X_train, y_train)

# ======================================
# Step 7: Predictions
# ======================================
y_pred = knn.predict(X_test)

# ======================================
# Step 8: Model Evaluation
# ======================================
accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)
```

```
print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
print(cm)

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

# ======================================
# Step 9: Confusion Matrix Visualization
# ======================================
plt.figure()

sns.heatmap(cm, annot=True)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()

# ======================================
# Step 10: Accuracy vs K Graph
# ======================================
k_values = range(1, 21)
accuracy_scores = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)

    pred = model.predict(X_test)
    accuracy_scores.append(accuracy_score(y_test, pred))

plt.figure()

plt.plot(k_values, accuracy_scores)

plt.xlabel("Value of K")
plt.ylabel("Accuracy")
plt.title("Accuracy vs K Value")

plt.show()

# ======================================
# Step 11: Custom Sample Prediction
# ======================================

# Create sample as DataFrame (to avoid warning)
sample_df = pd.DataFrame(
    [[5.1, 3.5, 1.4, 0.2]],
    columns=X.columns
)

# Scale sample
sample_scaled = scaler.transform(sample_df)
```

prediction = knn.predict(sample_scaled)

print("\nPrediction for sample [5.1, 3.5, 1.4, 0.2]:", prediction[0])

exp4 yogitaipynb

File Edit View Insert Runtime Tools Help

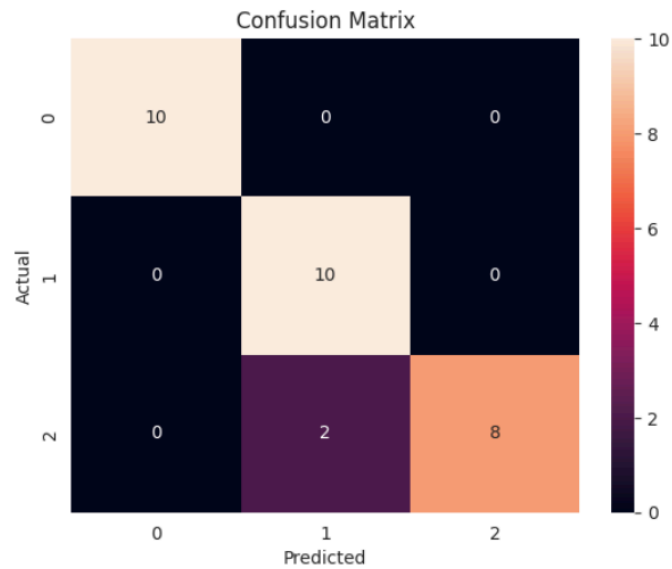
Commands + Code + Text Run all

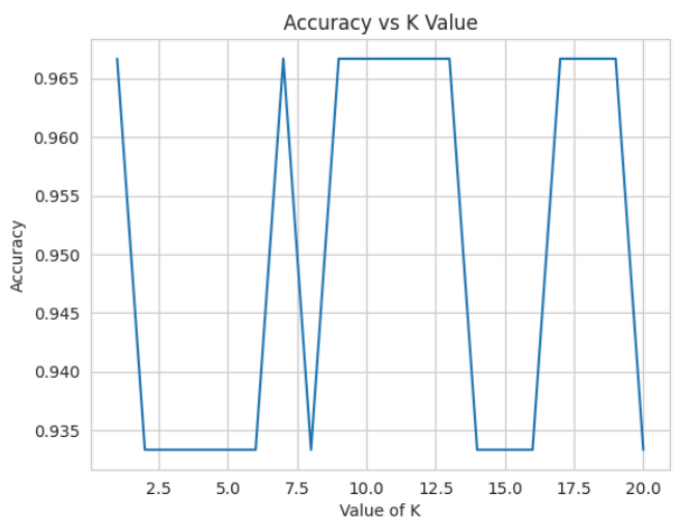
Choose Files Iris.csv

Iris.csv(text/csv) - 5107 bytes, last modified: 3/16/2026 - 100% done

```
Saving Iris.csv to Iris.csv
First 5 Rows:

   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa

Dataset Shape: (150, 6)

Feature Columns: ['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']
Target Classes: ['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']

Training Samples: 120
Testing Samples: 30

Model Accuracy: 0.9333333333333333

Confusion Matrix:
[[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]
```

```
Classification Report:
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       0.83      1.00      0.91        10
 Iris-virginica       1.00      0.80      0.89        10

       accuracy                           0.93        30
      macro avg       0.94      0.93      0.93        30
   weighted avg       0.94      0.93      0.93        30
```






```
Prediction for sample [5.1, 3.5, 1.4, 0.2]: Iris-setosa
```

Conclusion

The K-Nearest Neighbors (KNN) algorithm was implemented on the Iris dataset and achieved an accuracy of 93.33%. The model performed very well overall, with minor misclassification between Iris-versicolor and Iris-virginica due to overlapping features. The optimal value K = 5 provided a good balance between overfitting and underfitting.